

Practical 5: MNIST and CNNs in PyTorch

Data Engineering 414

Jacob Johannes Mostert

May 11, 2026

1 Introduction

This practical explores the implementation and evaluation of a Convolutional Neural Network (CNN) using the PyTorch framework. The primary objective is to build a model capable of accurately classifying handwritten digits from the MNIST dataset.

The core tasks require defining the convolutional and linear layers, and optimizing the network parameters through Stochastic Gradient Descent (SGD). By extracting and visualising the learned weights and activation maps, we aim to understand exactly how the network isolates distinct features to make its final classification decisions.

2 The Data

The initial phase of the practical required loading and verifying the MNIST dataset partitions. A predefined utility function extracted the feature vectors and corresponding target labels from compressed NumPy files. The dataset consists of images representing handwritten digits, which are flattened into 784-dimensional arrays.

3 Convolutional Neural Network Architecture

We built a Convolutional Neural Network (CNN) to extract visual features from the images and classify the digits. The network structure requires a 2D convolution layer, an activation function, and a final fully connected linear layer.

3.1 Dimensionality and Math

The first layer performs a 2D convolution. It takes an input image of height P_1 and width P_2 and slides a filter kernel of size $M_1 \times M_2$ across it. The amount the filter moves at each step is the stride (S). This operation produces an output feature map with new spatial dimensions, Q_1 and Q_2 . We calculate these new dimensions using the following formulas:

$$Q_1 = \left\lfloor \frac{P_1 - M_1}{S} \right\rfloor + 1 \tag{1}$$

$$Q_2 = \left\lfloor \frac{P_2 - M_2}{S} \right\rfloor + 1 \tag{2}$$

Our input images are 28 pixels by 28 pixels ($P_1 = 28, P_2 = 28$). We set the kernel size to 8 by 8 ($M_1 = 8, M_2 = 8$) and the stride to $S = 2$. Plugging these values into the equations gives us an output size of 11 by 11. We configured the convolutional layer to output 10 channels ($M_3 = 10$). To pass this data into the final linear layer, we must flatten it into a single one-dimensional array. The total number of features entering the dense layer is therefore $11 \times 11 \times 10 = 1210$.

3.2 Implementation

In the model class, we set up the ‘nn.Conv2d’ layer, the ‘nn.ReLU’ activation, and the ‘nn.Linear’ dense layer. The input data initially arrives as a flat array of 784 numbers. During the forward pass, we use the ‘view’ function to reshape this array back into a 28 by 28 grid so the 2D convolution can process it correctly. After the convolution and activation steps, we flatten the tensor again before feeding it to the dense layer to get our final predictions.

```

1 class CNN(nn.Module):
2     def __init__(self, M_1, M_2, M_3, K, stride):
3         super().__init__()
4
5         self.conv_1 = nn.Conv2d(in_channels=1, out_channels=M_3,
6 kernel_size=(M_1, M_2), stride=stride)
7         self.relu_1 = nn.ReLU()
8
9         Q1 = (28 - M_1) // stride + 1
10        Q2 = (28 - M_2) // stride + 1
11        dense_input_size = Q1 * Q2 * M_3
12
13        self.dense = nn.Linear(in_features=dense_input_size,
14 out_features=K)
15
16    def forward(self, x):
17        x_resaped = x.view(-1, 1, 28, 28)
18        z_1 = self.conv_1(x_resaped)
19        v_1 = self.relu_1(z_1)
20        v_1_flat = v_1.view(v_1.size(0), -1)
21
22        y_hat = self.dense(v_1_flat)
23
24        return y_hat, z_1, v_1

```

Listing 1: CNN Initialization and Forward Pass

With the class defined, we created an instance of the model. We also set up the cross-entropy loss function and the Stochastic Gradient Descent (SGD) optimizer.

```

1 model = CNN(M_1=8, M_2=8, M_3=10, K=10, stride=2)
2 loss_fn = nn.CrossEntropyLoss()
3 optimizer = torch.optim.SGD(model.parameters(), lr=lr)

```

Listing 2: Model Instantiation

4 Model Training and Optimization

Training the model involves making predictions, calculating the error, and updating the network weights to reduce that error. We measure the error using the cross-entropy loss

function, which compares the network’s predictions against the actual correct labels.

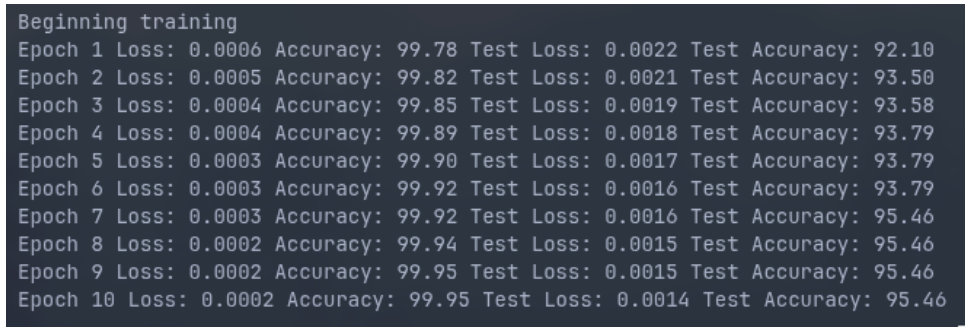
We update the weights using the SGD optimizer. The mathematical update rule takes the current weights θ_t , calculates the gradient of the loss $\nabla L(\theta_t)$, and scales it by a learning rate η to find the new weights θ_{t+1} :

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) \quad (3)$$

In PyTorch, we must explicitly clear the old gradients before running a new backward pass. If we do not clear them using `optimizer.zero_grad()`, the new gradients will add to the old ones and corrupt the training process.

```
1 # Training loop logic
2 optimizer.zero_grad()
3 y_hat, z_1, v_1 = model(X_i)
4 loss = loss_fn(y_hat, y_i)
5 loss.backward()
6 optimizer.step()
7
8 # Test loop logic
9 y_hat, z_1, v_1 = model(X_i)
10 loss = loss_fn(y_hat, y_i)
```

We ran this training process for 10 epochs. The loss steadily decreased while the accuracy increased, showing that the model successfully learned to identify the digits.



```
Beginning training
Epoch 1 Loss: 0.0006 Accuracy: 99.78 Test Loss: 0.0022 Test Accuracy: 92.10
Epoch 2 Loss: 0.0005 Accuracy: 99.82 Test Loss: 0.0021 Test Accuracy: 93.50
Epoch 3 Loss: 0.0004 Accuracy: 99.85 Test Loss: 0.0019 Test Accuracy: 93.58
Epoch 4 Loss: 0.0004 Accuracy: 99.89 Test Loss: 0.0018 Test Accuracy: 93.79
Epoch 5 Loss: 0.0003 Accuracy: 99.90 Test Loss: 0.0017 Test Accuracy: 93.79
Epoch 6 Loss: 0.0003 Accuracy: 99.92 Test Loss: 0.0016 Test Accuracy: 93.79
Epoch 7 Loss: 0.0003 Accuracy: 99.92 Test Loss: 0.0016 Test Accuracy: 95.46
Epoch 8 Loss: 0.0002 Accuracy: 99.94 Test Loss: 0.0015 Test Accuracy: 95.46
Epoch 9 Loss: 0.0002 Accuracy: 99.95 Test Loss: 0.0015 Test Accuracy: 95.46
Epoch 10 Loss: 0.0002 Accuracy: 99.95 Test Loss: 0.0014 Test Accuracy: 95.46
```

Figure 1: Training progress showing decreasing loss and increasing accuracy.

5 Feature Map and Filter Visualisation

To understand how the CNN processes images, we visualised the internal filters and output matrices for a single test image. We passed the first image from the test set through the model and extracted the weights from the convolutional layer.

The matrix $Z[1]$ represents the raw output directly after the convolution step. The matrix $V[1]$ shows the output after it passes through the ReLU activation function. The ReLU function works by converting all negative numbers to zero while leaving positive numbers unchanged. The formula is simply:

$$V[1] = \max(0, Z[1]) \quad (4)$$

This thresholding operation adds non-linearity to the network. It allows the model to ignore unimportant features and focus only on the most prominent patterns.

```

1 X = test_X[0,:]
2 x_input = X[None,:]
3
4 with torch.no_grad():
5     y_hat, z_1, v_1 = model(x_input)
6
7 H = model.conv_1.weight
8
9 plot_filters(x_input, H, z_1, v_1)

```

Listing 3: Extracting Weights and Visualising the First Example

The plotted filters reveal that the network searches for specific shapes and edges. The final feature maps show how certain areas of the image strongly activate specific filters while the ReLU function completely suppresses the rest.

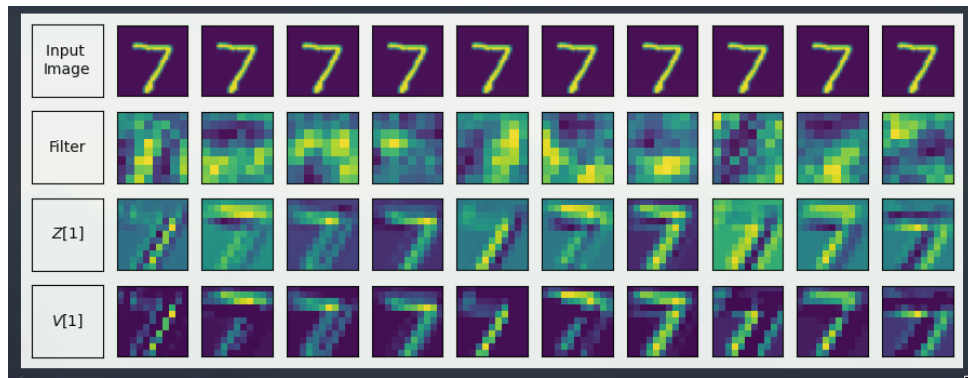


Figure 2: Terminal output displaying the dimensions of the loaded training and testing tensors.

6 Class-Specific Activations

To understand how the network differentiates between distinct shapes, we tested the model with three specific examples from our dataset. We selected indices 2, 3, and 5, representing the digits 1, 0, and 1.

6.1 Implementation Details

The code iterates through our chosen indices using a standard loop. For each iteration, we extract the corresponding 784-dimensional image vector from the test set.

A critical step before feeding this single image into the network is adjusting its shape. Because the CNN is designed to process batches of images simultaneously, it expects a four-dimensional input structure. We apply the `unsqueeze(0)` function to add a temporary batch dimension of size one. This ensures the geometric requirements of the convolution layer are met.

We then process the image inside a `torch.no_grad()` block. During training, PyTorch actively tracks every mathematical operation to calculate gradients for the backward pass. Since we are strictly evaluating the model here, calculating gradients is unnecessary. Disabling this tracking reduces memory consumption and speeds up the forward pass.

After the model generates its predictions (`y_hat`), we use `torch.argmax` to identify the specific class with the highest probability score. Finally, we pass the extracted con-

volutional weights (H), the raw feature maps (z_1), and the activated maps (v_1) into the `plot_filters` function to generate the visual grid.

```
1 indices_to_test = [2, 3, 5]
2
3 for idx in indices_to_test:
4     X_experiment = test_X[idx,:]
5
6     # Add a batch dimension for the CNN
7     X_input = X_experiment.unsqueeze(0)
8
9     # Disable gradient tracking for inference
10    with torch.no_grad():
11        y_hat, z_1, v_1 = model(X_input)
12
13    H = model.conv_1.weight
14
15    # Extract the highest probability prediction
16    predicted_class = torch.argmax(y_hat, dim=-1).item()
17    true_class = test_Y[idx].item()
18    print(f"Index {idx} | True Class: {true_class} | Predicted Class: {
19    predicted_class}")
20
21    plot_filters(X_experiment, H, z_1, v_1)
22    plt.show()
```

Listing 4: Testing Specific Digit Indices

6.2 Analysis of Filter Responses

The visual output confirms our model predicted the correct class for all three inputs. More importantly, the feature maps reveal the underlying mechanics of convolutional feature extraction.

Different digit shapes trigger distinct responses across the ten filter channels. When the network processes the digit '1', filters optimized for straight vertical lines shows high activation values. Conversely, when the network evaluates the digit '0', those same vertical filters remain dormant. Instead, channels attuned to rounded edges and curved structures become highly active. This selective excitation creates a unique activation signature for every number. The final linear layer relies entirely on these distinct signatures to classify the images accurately.

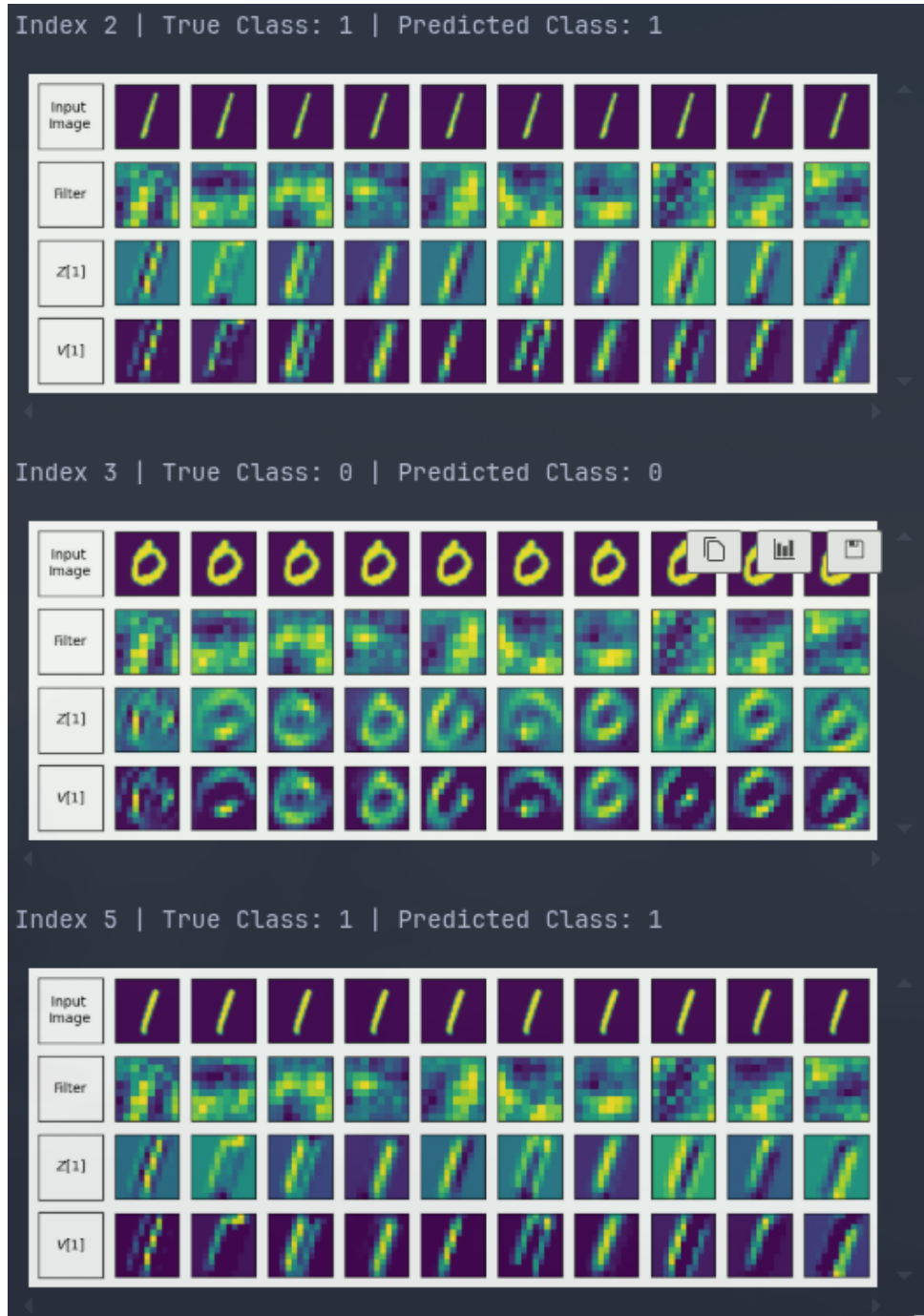


Figure 3: Visualisation of the convolutional filters and feature maps for the selected test images.

7 Conclusion

This practical provided a comprehensive exploration of Convolutional Neural Networks (CNNs) using the PyTorch framework. We successfully implemented a CNN architecture, trained it on the MNIST dataset, and visualised the learned filters and feature maps. The results demonstrated how the network extracts and processes visual features to classify handwritten digits accurately. By examining the activation patterns for specific digit examples, we gained insights into the internal workings of the CNN and how it differentiates

between various classes based on learned features.